RESULTLANG

Final Implementation and Demonstration

Presentation 3

CSE 303 - Compiler Design
Military Institute of Science and Technology

Md. Mahdi Israk Mahir 202414016

Rafsan Jony Asif 202414019

Instructor: Prof. Dr. Kazy Noor E Alam Siddiquee

Presentation 3 objectives

This milestone proves that ResultLang moves from source code to an executable program.

1. Show the implemented compiler modules and final language features
2. Demonstrate generated AST, symbol table, TAC and generated C
3. Build generated C with GCC and run program.exe
4. Report executed tests, error handling and final deliverables

Final language scope

Core compiler features

1. Identifiers
2. Numbers, strings and booleans
3. Assignment and arithmetic
4. Comparison expressions
5. if/else
6. show output

Domain features

1. Typed read for student name
2. Dynamic subject collection
3. Subject mark range 0 to 100
4. Total, average and grade
5. Result table generation
6. Runtime input validation

Out of scope: functions, arrays as user syntax, loops, objects and database integration.

Implementation structure


```
ResultLang/
  src/
    main.c
    lexer.l
    parser.y
    ast.c / ast.h
    semantic.c / semantic.h
    symbol_table.c / symbol_table.h
    tac.c / tac.h
    codegen.c / codegen.h
  examples/
  scripts/
  generated/
  outputs/
  report/
  presentation/
```

main.c	Verified runnable compiler driver
lexer.l and parser.y	Design artifacts for lexer and grammar
semantic and symbol table	Type checking and phase gates
tac and codegen	Intermediate code and C output
scripts	Windows build, compile and demo commands

Compiler pipeline




```
scripts\build.bat
scripts\compile_file.bat examples\valid\11_interactive_result.rl
generated\program.exe
python scripts\test_all.py
```

The same pipeline supports both scripted testing and the presentation-day live demo.

Lexer and parser implementation

Lexer responsibilities

1. Recognize identifiers, literals and keywords
2. Track line numbers for diagnostics
3. Reject unknown characters
4. Pass token values to parser rules

Parser responsibilities

1. Parse assignment, read, show and if/else
2. Apply operator precedence
3. Create AST nodes
4. Stop cleanly on syntax errors

```
read string student;
read subjects;
show result;
```

AST and symbol table


```
Program
  ReadString(student)
  ReadSubjects()
  ShowResult()
```

```
name,type,source
student,string,read
subject_count,number,internal
subject_names,string collection,internal
subject_marks,number collection,internal
total,number,internal
average,number,internal
grade,string,internal
```

AST records program structure. The symbol table records names, inferred types and generated internal values.

Semantic analysis

Check	Rule
Undefined variable	Identifier must exist before use
Type mismatch	Number, string and Boolean use must stay compatible
Invalid arithmetic	Arithmetic accepts numeric operands only
Invalid condition	if condition must evaluate to Boolean
Division by zero	Static zero denominators are rejected
Input range	Subject count and marks use runtime checks

Fatal semantic errors block TAC, C generation and executable creation.

Intermediate code generation


```
read.string student
read.number subject_count range 1..20
L_SUBJECT_INPUT:
read.string subject_names[i]
read.number subject_marks[i] range 0..100
total = total + subject_marks[i]
loop L_SUBJECT_INPUT
average = total / subject_count
grade = grade_from_average average
call show_result_table
```

TAC strategy

1. Keep IR readable for demonstration
2. Use labels for control flow
3. Preserve input validation rules
4. Separate compiler logic from generated C details

Generated C backend


```
rl_read_line("Enter student: ", rl_input_student,
sizeof(rl_input_student));
subject_count = rl_read_int_range("Enter number of subjects (1-20): ", 1,
20);


for (int rl_i = 0; rl_i < subject_count; rl_i++) {
    rl_read_line(rl_prompt, subject_names[rl_i],
sizeof(subject_names[rl_i]));
    subject_marks[rl_i] = rl_read_number_range(rl_prompt, 0, 100);
    total += subject_marks[rl_i];
}


average = rl_safe_div(total, subject_count);
grade = average >= 80 ? "A+" : ... : "F";
```

1. Uses C standard library only
2. Validates numeric input
3. Rejects marks outside 0 to 100
4. Builds a clean result table

Generated file:
generated/generated.c

Executable generation evidence


```
Building ResultLang compiler with
GCC...
Build complete:
build\resultlangc.exe

ResultLang compilation successful.
Outputs written to generated

Generated executable:
generated\program.exe
```

```
gcc -std=c11 -Wall -Wextra -O2 src\main.c -o
build\resultlangc.exe


build\resultlangc.exe
examples\valid\11_interactive_result.rl
generated

gcc -std=c11 -Wall -Wextra
generated\generated.c -o
generated\program.exe
```

The final executable is produced through the compiler output, not by hand-written C.

Live demonstration flow

1. Open examples\valid\11_interactive_result.rl
2. Run scripts\run_demo.bat
3. Enter student name and subject count
4. Enter subject names and marks
5. Show generated output and artifacts
6. Run one lexical, syntax and semantic error

```
scripts\run_demo.bat
```

Interactive demo output


```
Input:
Mahdi Israk
3
Compiler Design
85
Operating System
78
Database Systems
91
```

```
Output:
Result Summary
Mahdi Israk
Compiler Design 85.00
Operating System 78.00
Database Systems 91.00
Total 254.00
Average 84.67
Grade A+
```

The executable computes the final result from runtime input.

Testing strategy

Category	Verification method
Valid pipeline	Compile, generate C, build executable and compare output
Lexer tests	Unknown character and malformed string rejection
Parser tests	Missing semicolon, invalid expression and missing else
Semantic tests	Undefined name, type mismatch and invalid comparison
Runtime safety	Input range checks and division protection

Test results

21	21	0	11
tests executed	tests passed	tests failed	valid programs

Range	Purpose	Status
T01-T11	Valid compiler pipeline	PASS
T12-T13	Lexical rejection	PASS
T14-T15	Syntax rejection	PASS
T16-T21	Semantic rejection	PASS

Transcript: Build return code 0, Tests run 21, Tests passed 21, Tests failed 0.

Error handling evidence

Lexical

Unknown character
Malformed string

No generated executable

Syntax

Missing semicolon
Invalid expression
Missing else

No generated executable

Semantic

Undefined variable
Type mismatch
String arithmetic
Division by zero

No generated executable

Fatal compiler errors stop generated.c and program.exe creation.

Final report

10,000
verified report body words

APA
inline citation style

DOCX + PDF
editable and rendered files

1. Cover page, abstract, introduction and objectives
2. Language design, lexer, parser, AST and semantic analysis
3. TAC, C backend, build process and executable generation
4. Testing methodology, outcomes, limitations and appendices

Report files: report\ResultLang_Final_Report.docx and report\ResultLang_Final_Report.pdf

Final deliverables

Deliverable	Location
Compiler source	src, examples, scripts
Generated evidence	generated and outputs folders
Presentation	PPTX, PDF and speaker notes
Report	DOCX, PDF and word-count statement
Defense materials	Viva Q&A and questions for other groups

The package keeps presentation/report materials separate from source code so the repository can stay clean.

Strengths and limitations

Strengths

1. Complete compile to executable flow
2. Readable AST, symbol table and TAC
3. Interactive result demonstration
4. Automated valid and invalid tests

Limitations

1. No functions or user-defined arrays
2. No general-purpose loops in source syntax
3. Verified build uses self-contained C driver
4. Maximum 20 subjects in runtime collection

Live Demo Transition

ResultLang now compiles a small student-result language into C, builds an executable with GCC, and verifies behavior through automated tests.

```
cd E:\ResultLang
scripts\run_demo.bat
```

Ready for source, generated files, executable output and error demonstrations.